

1、说明：以下内容是基于我和朋友考了2次实操的经验，还有翻遍网友分享所总结出来的考点，所以具有非常高的参考价值。

2、考点：我发现无论是抽到简单题还是难题，它都是用鸢尾花数据集出题。当然它可能换其他数据集出题，但考点都是基于这4件套：划分特征X与标签y→划分为训练集和测试集→定义和训练模型→输出评估指标。它会在这4件套里做一些变形考点。

下面我以鸢尾花数据集为例，按【简单题】、【难题】、【难题变形】来细讲考点：

鸢尾花数据集包含150个鸢尾花样本，均匀分为山鸢尾、变色鸢尾和维吉尼亚鸢尾三个类别，每个样本测量了花萼长度、花萼宽度、花瓣长度和花瓣宽度四个连续数值特征。

```
In [52]: #导入鸢尾花数据集
import pandas as pd
from sklearn import datasets
data = datasets.load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
```

一、【简单题】考点：基本4件套，具体如下：

```
In [53]: #考点1：划分特征X与标签y，X只取前两个特征
X = df.iloc[:, 0:2]
y = df['target']

#考点2：按照8:2的比例，随机划分为训练集和测试集
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#考点3：定义逻辑回归模型，用划分后的数据在训练集上训练模型
from sklearn.linear_model import LogisticRegression
model = LogisticRegression() #我有朋友抽到考K近邻模型，则model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

#考点4：输出测试集的准确度
accuracy = model.score(X_test, y_test)
print(accuracy)

#考点5：输出测试集的预测值
```

```
y_pred = model.predict(X_test)
print(y_pred)
```

0.9

```
[1 0 2 1 2 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 1 0 2 2 2 2 0 0]
```

=====

二、【难题】考点：主要是 特征标准化、模型参数设置（比如迭代次数）、输出值的小数位要求、输出分类报告。具体如下：

In [54]: #考点1: 划分特征X与标签y

```
X = data.data
y = data.target
```

#考点2: 按照8:2的比例，随机划分为训练集和测试集

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

#考点3: 特征标准化

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

#考点4: 定义SVR模型，用划分后的数据在训练集上训练模型，模型迭代次数1000

```
from sklearn.svm import SVC
model = SVC(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)
```

#考点5: 输出测试集的准确度（要注意题目是否有小数位要求）

```
accuracy = model.score(X_test_scaled, y_test)
print(f'准确率: {accuracy}') #默认保留1位小数
print(f'准确率(保留2位小数): {accuracy:.2f}') #保留2位小数的写法
print(f'准确率(保留2位小数百分数): {accuracy:.2%}') #保留2位小数百分数的写法
```

#考点6: 输出测试集的预测值

```
y_pred = model.predict(X_test_scaled)
print(y_pred)
```

#考点7: 输出分类报告（要注意题目是否有小数位要求）

```

from sklearn.metrics import classification_report
report = classification_report(y_test, y_pred)
#默认保留2位小数, 若题目要求改小数位, 可添加digits参数, 如保留4位小数: report=classification_report(y_test, y_pred,digits=4)
print(report)

```

准确率: 1.0

准确率(保留2位小数): 1.00

准确率(保留2位小数百分数): 100.00%

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

=====

三、【难题变形】考点: 用pipeline将特征标准化和模型串联执行 (即将前述的考点3和4合并执行), 具体如下:

In [55]: #考点1: 划分特征X与标签y (同前)

```

X = data.data
y = data.target

```

#考点2: 按照8:2的比例, 随机划分为训练集和测试集 (同前)

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

#考点3-4串联执行: 特征标准化和模型串联执行

```

from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline

```

```

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(max_iter=1000, random_state=42))
])

```

```
] )  
  
pipe.fit(X_train, y_train)  
  
#考点5: 输出测试集的准确度  
accuracy = pipe.score(X_test, y_test)  
print(f'准确率: {accuracy}') #默认保留1位小数  
  
#考点6: 输出测试集的预测值  
y_pred = pipe.predict(X_test)  
print(y_pred)
```

准确率: 1.0

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

In []: